

Entropy and Information Theory

Table of Contents

1. [Introduction](#)
 2. [What is Information?](#)
 3. [Entropy Definition](#)
 4. [Properties of Entropy](#)
 5. [Conditional Entropy](#)
 6. [Mutual Information](#)
 7. [Practical Applications](#)
 8. [Computing Entropy in NumPy](#)
-

Introduction

Entropy measures uncertainty or randomness in a probability distribution. It's a fundamental concept in:

- Information theory
- Machine learning
- Statistical inference
- Data compression
- Decision making

Intuition: Entropy tells you how "surprised" you'd be on average when you observe an outcome.

What is Information?

Information Content

The **information content** (or surprise) of observing an outcome with probability p is:

$$I(x) = -\log_2(p(x)) \quad [\text{measured in bits}]$$

$$I(x) = -\ln(p(x)) \quad [\text{measured in nats}]$$

Intuition:

- **Rare events** (small p) give **lots of information** (high surprise)
- **Common events** (large p) give **little information** (low surprise)

Examples

```
import numpy as np

# Certain event (p = 1.0)
I = -np.log2(1.0) # = 0 bits (no surprise!)

# Coin flip (p = 0.5)
I = -np.log2(0.5) # = 1 bit (one yes/no question worth of info)

# Rare event (p = 0.01)
I = -np.log2(0.01) # ≈ 6.64 bits (very surprising!)

# Very rare event (p = 0.001)
I = -np.log2(0.001) # ≈ 9.97 bits (extremely surprising!)
```

Real-world interpretation:

- "The sun rose this morning" - $p \approx 1.0 \rightarrow 0$ bits (no surprise)
- "It rained today" - $p \approx 0.3 \rightarrow \sim 1.7$ bits (some surprise)
- "I won the lottery" - $p \approx 0.0000001 \rightarrow \sim 23$ bits (huge surprise!)

Why Logarithm?

Logarithms have nice properties:

1. **Additive for independent events:** $I(A \text{ and } B) = I(A) + I(B)$
2. **Monotonic:** lower probability $\rightarrow$ higher information
3. **Measures binary questions:** $\log_2$ counts yes/no questions needed

Entropy

Definition

Entropy is the **expected** (average) information content:

$$\begin{aligned} H(X) &= E[I(X)] = E[-\log p(X)] \\ &= \sum p(x) \cdot (-\log p(x)) \\ &= -\sum p(x) \cdot \log p(x) \end{aligned}$$

Base of logarithm:

- **Base 2:** Entropy measured in **bits**
- **Base e:** Entropy measured in **nats**
- **Base 10:** Entropy measured in **dits**

Most common: base 2 (bits)

Intuition

Entropy measures:

- **Uncertainty** in a distribution
- **Average surprise** when sampling
- **Minimum bits** needed to encode outcomes
- **Information gain** from observing the variable

Formula

For discrete random variable X with PMF $p(x)$:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Convention: $0 \cdot \log(0) = 0$ (by continuity, since $\lim_{p \rightarrow 0} p \log p = 0$)

Examples

Example 1: Certain outcome

$X = \{1\}$ with probability 1.0

$$H(X) = -1.0 \cdot \log_2(1.0) = 0 \text{ bits}$$

Interpretation: No uncertainty! We always know what will happen.

Example 2: Fair coin

$X = \{\text{Heads}, \text{Tails}\}$ with probabilities $[0.5, 0.5]$

$$\begin{aligned} H(X) &= -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) \\ &= -0.5 \cdot (-1) - 0.5 \cdot (-1) \\ &= 0.5 + 0.5 \\ &= 1 \text{ bit} \end{aligned}$$

Interpretation: Need exactly 1 bit to encode the outcome (0=Heads, 1=Tails).

Example 3: Biased coin

$X = \{\text{Heads}, \text{Tails}\}$ with probabilities $[0.9, 0.1]$

$$\begin{aligned} H(X) &= -0.9 \cdot \log_2(0.9) - 0.1 \cdot \log_2(0.1) \\ &= -0.9 \cdot (-0.152) - 0.1 \cdot (-3.322) \\ &= 0.137 + 0.332 \\ &= 0.469 \text{ bits} \end{aligned}$$

Interpretation: Less uncertainty than fair coin. Less than 1 bit on average.

Example 4: Fair 6-sided die

$X = \{1, 2, 3, 4, 5, 6\}$ with probabilities $[1/6, 1/6, 1/6, 1/6, 1/6, 1/6]$

$$\begin{aligned} H(X) &= -6 \cdot (1/6 \cdot \log_2(1/6)) \\ &= -6 \cdot (1/6 \cdot (-2.585)) \\ &= 2.585 \text{ bits} \end{aligned}$$

Interpretation: Need ~2.585 bits per roll on average.

Computing in NumPy

```
def entropy(pmf, base=2):  
    """  
    Compute entropy of a probability distribution  
  
    Parameters:  
    -----  
    pmf : array  
        Probability mass function  
    base : float  
        Logarithm base (2 for bits, np.e for nats)  
  
    Returns:  
    -----  
    H : float  
        Entropy  
    """  
    # Remove zeros ( $0 \log 0 = 0$  by convention)  
    p = pmf[pmf > 0]  
  
    # Compute entropy  
    if base == 2:  
        H = -np.sum(p * np.log2(p))  
    else:  
        H = -np.sum(p * np.log(p)) / np.log(base)  
  
    return H  
  
# Example: Fair die  
pmf = np.ones(6) / 6  
H = entropy(pmf)  
print(f"Entropy of fair die: {H:.3f} bits") # ~2.585 bits
```

Properties of Entropy

Property 1: Non-negative

$$H(X) \geq 0$$

Equality holds when X is deterministic (one outcome has probability 1).

Property 2: Maximum for Uniform Distribution

For a discrete variable with n possible outcomes:

$$H(X) \leq \log_2(n)$$

Equality holds when X is uniform: $p(x) = 1/n$ for all x .

Example:

```
# Maximum entropy for 16 outcomes
max_entropy = np.log2(16) # = 4 bits

# Achieved by uniform distribution
uniform = np.ones(16) / 16
H_uniform = entropy(uniform) # = 4 bits

# Any other distribution has lower entropy
biased = np.array([0.5] + [0.5/15]*15)
H_biased = entropy(biased) # < 4 bits

print(f"Max entropy: {max_entropy:.3f}")
print(f"Uniform: {H_uniform:.3f}")
print(f"Biased: {H_biased:.3f}")
```

Property 3: Concave Function

Entropy is a **concave** function of the probability distribution.

This means:

- Mixing distributions increases entropy
- Sharper distributions have lower entropy
- Uniform is the "flattest" and has maximum entropy

Property 4: Invariant to Permutation

Reordering outcomes doesn't change entropy:

```
pmf1 = np.array([0.5, 0.3, 0.2])
pmf2 = np.array([0.2, 0.5, 0.3])

assert np.isclose(entropy(pmf1), entropy(pmf2))
```

Conditional Entropy

Definition

Conditional entropy $H(Y|X)$ is the average uncertainty in Y when X is known:

$$\begin{aligned} H(Y | X) &= \sum p(x) \cdot H(Y | X=x) \\ &= \sum p(x) \cdot [-\sum p(y|x) \log p(y|x)] \\ &= -\sum \sum p(x, y) \log p(y|x) \end{aligned}$$

Intuition: "How much uncertainty remains in Y after observing X ?"

Example: Submarine Location

Question: If we know the y-coordinate, how much uncertainty remains in the x-coordinate?

```

# For each y value, compute entropy of P(X | Y=y)
def conditional_entropy_x_given_y(joint_pmf):
    """
    Compute  $H(X | Y)$ 

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
        Joint probability distribution

    Returns:
    -----
    H_X_given_Y : float
        Conditional entropy
    """
    # Marginal P(Y)
    p_y = np.sum(joint_pmf, axis=0)

    # Conditional PMF P(X | Y)
    # Avoid division by zero
    p_y_safe = np.where(p_y > 0, p_y, 1.0)
    p_x_given_y = joint_pmf / p_y_safe[None, :]

    # Entropy of each conditional distribution
    H_conditional = np.zeros(joint_pmf.shape[1])
    for y in range(joint_pmf.shape[1]):
        if p_y[y] > 0:
            H_conditional[y] = entropy(p_x_given_y[:, y])

    # Weighted average
    H_X_given_Y = np.sum(p_y * H_conditional)

    return H_X_given_Y

# Example
H_X_given_Y = conditional_entropy_x_given_y(submarine_pmf)
print(f"H(X | Y) = {H_X_given_Y:.3f} bits")

```

Conditional Entropy for Specific Value

Question: If we know $y=4$, what's the entropy of x ?

```
# Conditional PMF P(X | Y=4)
p_x_given_y4 = submarine_pmf[:, 4] / np.sum(submarine_pmf[:, 4])

# Entropy of this distribution
H_X_given_y4 = entropy(p_x_given_y4)
print(f"H(X | Y=4) = {H_X_given_y4:.3f} bits")
```

Properties of Conditional Entropy

Chain Rule:

$$H(X, Y) = H(Y) + H(X | Y) = H(X) + H(Y | X)$$

Intuition: Total uncertainty = uncertainty in first variable + remaining uncertainty in second

Non-negative:

$$H(X | Y) \geq 0$$

Information reduces uncertainty:

$$H(X | Y) \leq H(X)$$

Knowing Y can only reduce (or keep same) uncertainty in X , never increase it!

Equality holds when X and Y are independent.

Mutual Information

Definition

Mutual information $I(X; Y)$ measures how much information X and Y share:

$$\begin{aligned} I(X; Y) &= H(X) - H(X | Y) \\ &= H(Y) - H(Y | X) \\ &= H(X) + H(Y) - H(X, Y) \end{aligned}$$

Intuition: "How much does knowing X reduce uncertainty in Y ?" (or vice versa)

Example

```
def mutual_information(joint_pmf):
    """
    Compute  $I(X; Y)$ 

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
        Joint probability distribution

    Returns:
    -----
    I_XY : float
        Mutual information
    """
    # Marginals
    p_x = np.sum(joint_pmf, axis=1)
    p_y = np.sum(joint_pmf, axis=0)

    # Marginal entropies
    H_X = entropy(p_x)
    H_Y = entropy(p_y)

    # Joint entropy
    H_XY = entropy(joint_pmf.ravel())

    # Mutual information
    I_XY = H_X + H_Y - H_XY

    return I_XY

# Example
I_XY = mutual_information(submarine_pmf)
H_X = entropy(np.sum(submarine_pmf, axis=1))
H_Y = entropy(np.sum(submarine_pmf, axis=0))

print(f"H(X) = {H_X:.3f} bits")
print(f"H(Y) = {H_Y:.3f} bits")
```

```
print(f"I(X; Y) = {I_XY:.3f} bits")  
print(f"Fraction of X explained by Y: {I_XY / H_X:.2%}")
```

Properties of Mutual Information

Symmetric:

$$I(X; Y) = I(Y; X)$$

Non-negative:

$$I(X; Y) \geq 0$$

Zero iff independent:

$$I(X; Y) = 0 \iff X \text{ and } Y \text{ are independent}$$

Bounded:

$$I(X; Y) \leq \min(H(X), H(Y))$$

Applications

Application 1: Feature Selection

Question: Which y-coordinate gives most information about x-coordinate?

```
# For each y value, compute H(X | Y=y)
entropies = np.zeros(16)
p_y = np.sum(submarine_pmf, axis=0)

for y in range(16):
    if p_y[y] > 0:
        p_x_given_y = submarine_pmf[:, y] / p_y[y]
        entropies[y] = entropy(p_x_given_y)
    else:
        entropies[y] = np.inf # Undefined

# Find y with minimum entropy (most informative)
most_informative_y = np.argmin(entropies)
print(f"Most informative y: {most_informative_y}")
print(f"Entropy: {entropies[most_informative_y]:.3f} bits")

# Compare to marginal entropy
H_X = entropy(np.sum(submarine_pmf, axis=1))
print(f"Original H(X): {H_X:.3f} bits")
print(f"Reduction: {H_X - entropies[most_informative_y]:.3f} bits")
```

Application 2: Decision Trees

Entropy is used in decision tree algorithms (ID3, C4.5) to select which feature to split on:

1. Compute entropy of current node
2. For each feature, compute conditional entropy after split
3. Choose feature with maximum information gain (reduction in entropy)

Application 3: Data Compression

Entropy gives a lower bound on average bits needed to encode data:

Shannon's Source Coding Theorem:

Expected code length $\geq H(X)$

Example:

```
# PMF of English letters
pmf_letters = np.array([0.08, 0.015, 0.03, ...]) # Approximate frequencies

H = entropy(pmf_letters, base=2)
print(f"Minimum bits per letter: {H:.2f}")

# Fixed-length encoding (ASCII)
fixed_length = np.log2(26) # 4.7 bits per letter
print(f"Fixed-length (ASCII): {fixed_length:.2f} bits")

# Huffman coding achieves close to H
```

Application 4: Model Comparison

Lower entropy posterior = more confident model

```
# Compare two models
prior1 = submarine_pmf
prior2 = np.ones((16, 16)) / 256 # Uniform

likelihood = search_submarine(10, 7)

posterior1 = (likelihood * prior1) / np.sum(likelihood * prior1)
posterior2 = (likelihood * prior2) / np.sum(likelihood * prior2)

H1 = entropy(posterior1.ravel())
H2 = entropy(posterior2.ravel())

print(f"Entropy of posterior 1: {H1:.3f} bits")
print(f"Entropy of posterior 2: {H2:.3f} bits")

if H1 < H2:
    print("Model 1 is more confident")
else:
    print("Model 2 is more confident")
```

Computing Entropy in NumPy

Pattern 1: Entropy of 1D Distribution

```
def entropy(pmf, base=2):  
    """Compute entropy of a discrete distribution"""  
    # Remove zeros  
    p = pmf[pmf > 0]  
  
    # Compute entropy  
    if base == 2:  
        return -np.sum(p * np.log2(p))  
    elif base == np.e:  
        return -np.sum(p * np.log(p))  
    else:  
        return -np.sum(p * np.log(p)) / np.log(base)  
  
# Example  
pmf = np.array([0.5, 0.3, 0.2])  
H = entropy(pmf)  
print(f"H = {H:.3f} bits")
```

Pattern 2: Entropy of 2D Joint Distribution

```
# Flatten to 1D  
joint_pmf = submarine_pmf  
H_joint = entropy(joint_pmf.ravel())  
print(f"H(X, Y) = {H_joint:.3f} bits")
```

Pattern 3: Conditional Entropy $H(X | Y)$

```
def conditional_entropy(joint_pmf, axis=0):
    """
    Compute conditional entropy

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
    axis : int
        0: compute  $H(X | Y)$  [entropy of rows given columns]
        1: compute  $H(Y | X)$  [entropy of columns given rows]

    Returns:
    -----
    H : float
        Conditional entropy
    """
    if axis == 0:
        #  $H(X | Y)$ : for each y, compute entropy of  $p(x|y)$ , then average
        p_y = np.sum(joint_pmf, axis=0) # Marginal P(Y)
        p_x_given_y = joint_pmf / np.where(p_y > 0, p_y, 1.0)[None, :]

        H_conditional = np.zeros(joint_pmf.shape[1])
        for y in range(joint_pmf.shape[1]):
            if p_y[y] > 0:
                H_conditional[y] = entropy(p_x_given_y[:, y])

        return np.sum(p_y * H_conditional)

    else:
        #  $H(Y | X)$ : for each x, compute entropy of  $p(y|x)$ , then average
        p_x = np.sum(joint_pmf, axis=1) # Marginal P(X)
        p_y_given_x = joint_pmf / np.where(p_x > 0, p_x, 1.0)[: , None]

        H_conditional = np.zeros(joint_pmf.shape[0])
        for x in range(joint_pmf.shape[0]):
            if p_x[x] > 0:
                H_conditional[x] = entropy(p_y_given_x[x, :])
```

```
        return np.sum(p_x * H_conditional)

# Example
H_X_given_Y = conditional_entropy(submarine_pmf, axis=0)
H_Y_given_X = conditional_entropy(submarine_pmf, axis=1)
print(f"H(X | Y) = {H_X_given_Y:.3f} bits")
print(f"H(Y | X) = {H_Y_given_X:.3f} bits")
```

Pattern 4: Mutual Information

```
def mutual_information(joint_pmf):
    """Compute I(X; Y)"""
    # Marginals
    p_x = np.sum(joint_pmf, axis=1)
    p_y = np.sum(joint_pmf, axis=0)

    # Entropies
    H_X = entropy(p_x)
    H_Y = entropy(p_y)
    H_XY = entropy(joint_pmf.ravel())

    # Mutual information
    return H_X + H_Y - H_XY

# Example
I = mutual_information(submarine_pmf)
print(f"I(X; Y) = {I:.3f} bits")
```

Pattern 5: Entropy of Each Conditional Distribution

```
def entropy_each_conditional(joint_pmf, axis=0):
    """
    Compute entropy of each conditional distribution

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
    axis : int
        0: compute  $H(X \mid Y=y)$  for each y
        1: compute  $H(Y \mid X=x)$  for each x

    Returns:
    -----
    entropies : array
        Entropy of each conditional distribution
    """
    if axis == 0:
        # For each y, compute  $H(X \mid Y=y)$ 
        p_y = np.sum(joint_pmf, axis=0)
        p_x_given_y = joint_pmf / np.where(p_y > 0, p_y, 1.0)[None, :]

        entropies = np.zeros(joint_pmf.shape[1])
        for y in range(joint_pmf.shape[1]):
            if p_y[y] > 0:
                entropies[y] = entropy(p_x_given_y[:, y])
            else:
                entropies[y] = np.nan # Undefined

        return entropies

    else:
        # For each x, compute  $H(Y \mid X=x)$ 
        p_x = np.sum(joint_pmf, axis=1)
        p_y_given_x = joint_pmf / np.where(p_x > 0, p_x, 1.0)[: , None]

        entropies = np.zeros(joint_pmf.shape[0])
        for x in range(joint_pmf.shape[0]):
            if p_x[x] > 0:
```

```
        entropies[x] = entropy(p_y_given_x[x, :])
    else:
        entropies[x] = np.nan # Undefined

    return entropies

# Example: Find which y gives most information about x
entropy_y = entropy_each_conditional(submarine_pmf, axis=0)
most_informative_y = np.nanargmin(entropy_y)
print(f"Most informative y: {most_informative_y}")
print(f"H(X | Y={most_informative_y}) = {entropy_y[most_informative_y]:.3f} bi
```

Common Mistakes

Mistake 1: Not handling zeros

```
# WRONG: log(0) = -inf
pmf = np.array([0.5, 0.5, 0.0, 0.0])
H = -np.sum(pmf * np.log2(pmf)) # Returns nan or -inf

# CORRECT: Filter out zeros first
p = pmf[pmf > 0]
H = -np.sum(p * np.log2(p)) # Works correctly
```

Mistake 2: Forgetting to normalize

```
# WRONG: Not a valid PMF
values = np.array([1, 2, 3])
H = entropy(values) # Sum is 6, not 1!

# CORRECT: Normalize first
pmf = values / np.sum(values)
H = entropy(pmf)
```

Mistake 3: Using natural log instead of log2

```
# Gives different units!
H_bits = -np.sum(p * np.log2(p)) # In bits
H_nats = -np.sum(p * np.log(p))  # In nats

# Conversion: nats = bits / log(2)
assert np.isclose(H_nats, H_bits / np.log(2))
```

Mistake 4: Confusing joint and conditional entropy

```
#  $H(X, Y) \neq H(X | Y)$ 

# Joint entropy
H_XY = entropy(joint_pmf.ravel()) # Uncertainty in both

# Conditional entropy
H_X_given_Y = conditional_entropy(joint_pmf, axis=0) # Remaining uncertainty
```

Summary

Entropy $H(X)$:

$$H(X) = -\sum p(x) \log_2 p(x) \quad [\text{bits}]$$

- Measures uncertainty/randomness
- Minimum: 0 (deterministic)
- Maximum: $\log_2(n)$ (uniform over n outcomes)

Conditional Entropy $H(X|Y)$:

$$H(X | Y) = \sum p(y) H(X | Y=y)$$

- Average uncertainty in X given Y
- Always $\leq H(X)$ (information reduces uncertainty)

Mutual Information $I(X;Y)$:

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X)$$

- Information shared between X and Y
- Symmetric: $I(X;Y) = I(Y;X)$
- Zero iff independent

Key Relationships:

$$\begin{aligned} H(X, Y) &= H(X) + H(Y | X) = H(Y) + H(X | Y) \\ I(X; Y) &= H(X) + H(Y) - H(X, Y) \end{aligned}$$

Applications:

- Feature selection (choose feature with max information gain)

- Decision trees (split on feature minimizing conditional entropy)
- Data compression (entropy is minimum average bits needed)
- Model evaluation (lower entropy = more confident predictions)

NumPy Tips:

- Filter out zeros before computing log
- Use base 2 for bits, base e for nats
- Verify PMF sums to 1 before computing entropy
- Handle edge cases (empty arrays, all zeros, etc.)